Predicción de fallas en una flota de máquinas mediante deep learning

Proyecto del curso Machine Learning y Deep Learning

Profesor: Enrique García/Gildardo Sanchez · Duración: 3 semanas (≈ 70 horas)

1. Contexto

Una planta opera una flota de 100 máquinas rotativas de cuatro modelos distintos. Cada máquina tiene cuatro componentes críticos (`comp1` a `comp4`). Cuando un componente falla sin aviso, la máquina sale de operación de forma no programada: se pierde producción, se moviliza al técnico de guardia y, con frecuencia, se daña material en proceso. El departamento de mantenimiento quiere pasar de una estrategia *correctiva* y *por calendario* a una estrategia *predictiva*: anticipar con suficiente tiempo qué máquina va a fallar, y en qué componente, para intervenirla en una ventana planeada.

Ustedes forman parte del equipo de analítica de la planta. Su tarea es construir, evaluar y justificar ante la gerencia un sistema de alerta temprana basado en deep learning.

Perspectiva de planta

En planta, un modelo no se juzga por su exactitud, sino por las decisiones que permite tomar: cuántas fallas evita, cuántas falsas alarmas genera y con cuántas horas de anticipación avisa. Tengan eso en mente desde el primer día.

2. Objetivos de aprendizaje

Al terminar el proyecto, cada estudiante será capaz de:

1. Integrar varias fuentes de datos industriales (telemetría, eventos, mantenimiento, metadatos) en un tensor de entrada coherente para modelos secuenciales.
2. Definir correctamente una etiqueta de predicción con horizonte temporal y detectar fugas de información (*data leakage*) en series de tiempo.
3. Diseñar, entrenar y comparar arquitecturas de deep learning para secuencias (LSTM/GRU, CNN 1D, TCN, Transformer) que combinen entradas dinámicas y estáticas.
4. Manejar desbalance extremo de clases y evaluar con métricas adecuadas (PR-AUC, recall a precisión fija, tiempo de anticipación).
5. Cuantificar incertidumbre, calibrar probabilidades e interpretar modelos.
6. Traducir el desempeño del modelo a una política de mantenimiento con impacto económico.

3. Datos

Se usará el conjunto público **Microsoft Azure Predictive Maintenance**, el cual es un conjunto simulado por Microsoft con fines didácticos; cubre el año 2015 (los registros de mantenimiento incluyen además parte de 2014).

Archivo	Columnas	Descripción
`PdM_telemetry.csv`	`datetime, machineID, volt, rotate, pressure, vibration`	Promedio horario de cuatro sensores por máquina. Es la fuente principal (∼876 000 filas).
`PdM_errors.csv`	`datetime, machineID, errorID`	Errores no fatales registrados por el controlador (`error1`–`error5`).
`PdM_maint.csv`	`datetime, machineID, comp`	Reemplazos de componentes, tanto programados como posteriores a una falla.
`PdM_failures.csv`	`datetime, machineID, failure`	Reemplazos causados por falla, indicando el componente que falló.
`PdM_machines.csv`	`machineID, model, age`	Metadatos estáticos: modelo de máquina y antigüedad en años.

Tabla 1: Archivos del conjunto de datos.

4. Definición del problema

Para cada máquina m y cada hora t, el modelo recibe la historia observada hasta t y debe estimar:

- **Tarea principal (obligatoria).** La probabilidad de que la máquina presente una falla en cualquier componente en el intervalo $(t,\, t+H]$, con $H = 24$ horas.
- **Tarea secundaria (obligatoria a partir de la semana 2).** Cuál componente fallará: clasificación en cinco clases $\{\text{ninguno}, \texttt{comp1}, \ldots, \texttt{comp4}\}$.

Partición temporal fija

Para que los resultados de todos los equipos sean comparables, se usará esta partición. No se permite partición aleatoria por filas.

Conjunto	Periodo de las horas t a predecir	Uso
Entrenamiento	1 ene 2015 – 31 ago 2015	Ajuste de parámetros
Validación	1 sep 2015 – 31 oct 2015	Selección de modelo e hiperparámetros
Prueba	1 nov 2015 – 31 dic 2015	Solo para el reporte final (semana 3)

Tabla 2: Partición temporal obligatoria.

Las ventanas de entrada pueden tomar datos anteriores al inicio de cada periodo, pero ninguna etiqueta de entrenamiento puede depender de eventos posteriores al 31 de agosto, y ninguna de validación de eventos posteriores al 31 de octubre. Dejen un margen de H horas en cada frontera.

Atención

El conjunto de prueba se usa **una sola vez**, en la semana 3, con el modelo ya elegido. Si un equipo ajusta decisiones mirando el desempeño en prueba, el resultado reportado no es válido y se penalizará en la rúbrica.

5. Reglas generales

- Equipos como están conformados hasta ahora. La carga estimada es de ≈ 70 horas por persona.

- Lenguaje Python. Para deep learning se puede usar PyTorch o TensorFlow/Keras.
- Todo debe ser reproducible: repositorio con `README`, archivo de dependencias, semillas fijas y un script o notebook que regenere las cifras reportadas.
- Se pueden consultar artículos, documentación y notebooks públicos, pero deben citarse. Copiar una solución existente sin atribución se considera plagio.
- Cada entrega incluye un reporte breve en PDF y el estado del repositorio en esa fecha (*tag* de git: `entrega1`, `entrega2`, `entrega3`).

6. Plan de trabajo y entregas semanales

6.1. Semana 1: integración de datos, etiquetado y línea base (≈ 22 h)

1.1 **Análisis exploratorio** (5 h). Distribución de cada sensor por modelo de máquina; frecuencia de errores, mantenimientos y fallas por componente; comportamiento de la telemetría en las 48–72 horas previas a una falla frente a periodos sanos.

1.2 **Análisis de los registros de mantenimiento** (3 h). Verificar cómo se relacionan `PdM_maint` y `PdM_failures`. Responder: ¿qué registros de mantenimiento son consecuencia de una falla y qué riesgo implica usarlos como variable de entrada?

1.3 **Construcción de etiquetas** (4 h). Generar la etiqueta binaria y la multiclase para $H = 24$ h. Reportar la proporción de positivos por conjunto y por componente.

1.4 **Ingeniería de variables para la línea base** (5 h). Promedios y desviaciones móviles (3 h y 24 h), conteo de errores en las últimas 24 h, horas desde el último reemplazo de cada componente, modelo y antigüedad.

1.5 **Línea base clásica** (5 h). Regresión logística y un modelo de *gradient boosting* (XGBoost o LightGBM) sobre esas variables. Evaluar en validación.

Entrega 1 — [Vie 13 Nov]

- Reporte de máximo 5 páginas con: hallazgos del análisis exploratorio (3 a 5 gráficas relevantes), definición de la etiqueta, tabla de desbalance, análisis de fuga de información y resultados de la línea base.
- Métricas en validación: PR-AUC, ROC-AUC, recall con precisión ≥ 0.80 y matriz de confusión multiclase.
- Repositorio con el *pipeline* de datos que produzca los conjuntos de entrenamiento y validación desde los CSV originales.

6.2. Semana 2: modelos secuenciales de deep learning (≈ 25 h)

2.1 **Diseño del tensor de entrada** (5 h). Para cada (m, t), una ventana de L horas (explorar al menos $L \in \{24, 72, 168\}$) con canales dinámicos: los cuatro sensores normalizados, indicadores horarios de cada tipo de error y horas desde el último reemplazo de cada componente. Entradas estáticas: *embedding* del modelo de máquina y antigüedad. Justificar la normalización (global, por modelo o por máquina) y evitar calcular estadísticas con datos de validación.

2.2 **Arquitecturas** (10 h). Implementar y comparar al menos tres:

- LSTM o GRU (una o dos capas) con las entradas estáticas concatenadas al estado final.
- CNN 1D o TCN (convoluciones causales dilatadas).
- Un híbrido CNN-LSTM.

Todas con dos salidas (binaria y multiclase) y pérdida combinada; justificar la ponderación.

2.3 **Desbalance de clases** (5 h). Comparar al menos dos estrategias: pérdida ponderada, *focal loss*, submuestreo de horas sanas, sobremuestreo de ventanas pre-falla. Explicar cómo afecta cada una a la calibración de las probabilidades.

2.4 **Entrenamiento disciplinado** (5 h). *Early stopping* sobre PR-AUC de validación, regularización (*dropout*, *weight decay*), búsqueda de hiperparámetros con un presupuesto declarado (por ejemplo, 30 ensayos con Optuna). Curvas de aprendizaje.

Entrega 2 — [Vie 20 Nov]

- Reporte de máximo 6 páginas con: diagrama del tensor de entrada y de cada arquitectura, tabla comparativa en validación contra la línea base de la semana 1, efecto de la longitud de ventana L y de la estrategia de desbalance.
- Métricas adicionales: F1 macro por componente y **tiempo de anticipación**, definido como las horas entre la primera alerta sostenida (probabilidad sobre el umbral durante al menos 3 horas seguidas) y la falla real.
- Discusión: ¿supera el deep learning a la línea base? Si no, ¿por qué? Una respuesta honesta y bien argumentada vale más que una mejora marginal sin explicación.

6.3. Semana 3: modelo avanzado, confiabilidad y política de mantenimiento (≈ 23 h)

3.1 **Modelo avanzado** (6 h). Elegir **una** opción:

(a) Codificador Transformer con codificación posicional y *token* de máquina.

(b) Autoencoder LSTM entrenado solo con ventanas sanas, usando el error de reconstrucción como índice de salud; comparar contra el modelo supervisado.

(c) Modelo de tiempo hasta la falla por componente (regresión con tope o enfoque de supervivencia discreta).

3.2 **Incertidumbre y calibración** (4 h). MC Dropout o ensamble de 5 modelos; diagrama de confiabilidad y *Brier score*; calibración con *temperature scaling* si es necesario.

3.3 **Interpretabilidad** (3 h). *Integrated gradients*, SHAP o saliencia temporal: ¿qué canales y qué horas de la ventana impulsan las alertas de cada componente? Contrastar con lo observado en el análisis exploratorio.

3.4 **Política de mantenimiento** (6 h). Con los costos de la Tabla 3, elegir el umbral de alerta que minimice el costo esperado en **validación**, y después evaluar esa política una sola vez en **prueba**. Comparar contra dos referencias: operar hasta la falla, y reemplazo por calendario de cada componente.

3.5 **Comunicación** (4 h). Reporte final y presentación.

Concepto	Costo (unidades monetarias)
Falla no anticipada (paro no programado, reparación, material dañado)	15 000
Intervención preventiva acertada (falla real en las siguientes 24 h)	3 000
Falsa alarma (inspección sin falla, paro planeado breve)	600

Tabla 3: Supuestos de costo para el análisis económico. Cada equipo debe además hacer un análisis de sensibilidad variando la razón entre el costo de falla y el de falsa alarma.

Entrega 3 (final) — [Vie 27 Nov]

- Reporte técnico de máximo 10 páginas (más anexos) que integre las tres semanas, con resultados finales en el conjunto de prueba.
- Tabla de costo anual estimado para las tres políticas y análisis de sensibilidad.
- Presentación de 10 minutos dirigida al gerente de planta: sin jerga, centrada en fallas evitadas, falsas alarmas por semana, anticipación promedio y ahorro estimado. Incluir limitaciones y qué se necesitaría para llevar el modelo a producción.
- Repositorio final reproducible (*tag* `entrega3`).

7. Evaluación

Componente	Peso	Qué se evalúa
Entrega 1	20 %	Integración de datos correcta, etiquetas sin fuga, línea base sólida y análisis exploratorio con hallazgos útiles.
Entrega 2	30 %	Diseño del tensor, variedad y rigor en la comparación de arquitecturas, manejo del desbalance, discusión crítica de resultados.
Entrega 3: modelado avanzado	15 %	Modelo avanzado, incertidumbre, calibración e interpretabilidad.
Entrega 3: política y negocio	20 %	Umbral elegido sin tocar prueba, comparación con políticas de referencia, análisis de sensibilidad.
Comunicación y reproducibilidad	15 %	Claridad de reportes y presentación; el repositorio regenera las cifras reportadas.

Tabla 4: Rúbrica general.

Penalizaciones. Uso del conjunto de prueba antes de la semana 3 o fuga de información no detectada: hasta −20 % de la nota del proyecto.

8. Recomendaciones

- Empiecen por un modelo simple que funcione de punta a punta; la complejidad se agrega después.
- La exactitud (*accuracy*) no es informativa con menos del 2 % de positivos. No la reporten como métrica principal.
- Revisen a mano algunas máquinas: grafiquen la probabilidad predicha junto con los errores, mantenimientos y fallas reales. Muchos problemas se descubren así antes que en una tabla.

- El conjunto es simulado y algunos patrones pueden ser más limpios que en una planta real. Comenten qué esperarían que cambiara con datos reales (deriva de sensores, datos faltantes, registros de mantenimiento incompletos).

Referencias sugeridas

- Microsoft. *Predictive Maintenance Modelling Guide* (Azure AI Gallery), documento que acompaña al conjunto de datos.
- Hochreiter, S. y Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*.
- Bai, S., Kolter, J. Z. y Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv:1803.01271*.
- Lin, T.-Y. et al. (2017). Focal Loss for Dense Object Detection. *ICCV*.
- Gal, Y. y Ghahramani, Z. (2016). Dropout as a Bayesian Approximation. *ICML*.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*.